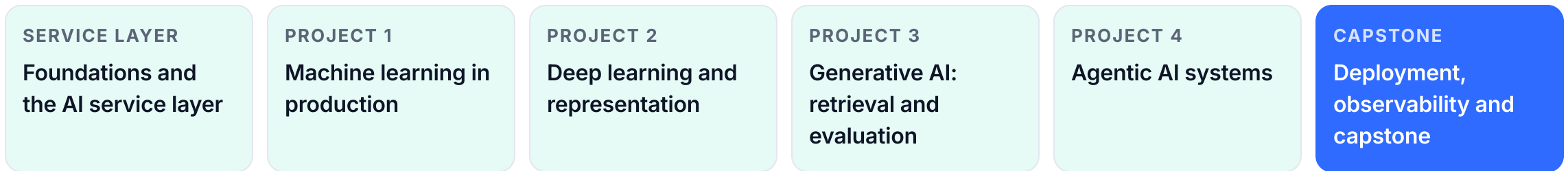


Deployment, observability and capstone

AI ENGINEERING: GENAI AND AGENTIC SYSTEMS

Today you put Project 1 in front of the agent as a cheap pre-filter and measure cost per resolved ticket before and after. Then you promote an approved artifact to the live target behind a canary, release signed images through the delivery pipeline with a human approval, and write down who or what takes a bad release back on your track.

The system runs; now the organisation asks who approved it and what it costs



Four services and an agent run on the platform, each registered and behind the model gateway. Nothing in this part deploys from your laptop except through the platform's own commands: the registry, the delivery pipeline, the canary.

Today you promote, deliver and price the whole system

- 1 Where you stand on the platform, and what it costs
- 2 Route cheap first
- 3 Cost per resolved ticket, before and after
- 4 The promotion drill: approve, go live behind a canary
- 5 Images through the delivery pipeline, and who rolls back
- 6 Dashboards, SLOs and the exported metrics
- 7 Lower and higher environments
- 8 The part's check, demos, debrief

One file holds the numbers

[sessions/session-06/NOTES.md](#) : three cost sheet figures, two costs per resolution and the gateway spend, the promotion, the rollback command, one SLO row.

The part's check reads it and asks for what is missing.

The EU AI Act applies now; the Omnibus deferred the high-risk dates

Date	What applies
2024-08-01	the Act enters into force
2025-02-02	prohibited practices, Article 5
2025-08-02	general-purpose AI provider obligations
2026-07-27	the Digital Omnibus on AI enters into force
2026-08-02	general application, with the Article 50 transparency rules
2027-12-02	high-risk in Annex III areas: biometrics, critical infrastructure, education, employment, credit, migration
2028-08-02	high-risk AI embedded in regulated products, Annex I

Cost is a design input, and cost per resolution is the number that matters

98%

of FinOps practitioners now manage AI spend: up from 63 percent in 2025 and 31 percent in 2024 (State of FinOps 2026)

- **Prices per capability fall** (the first part's landscape has the curve), and agents spend far more tokens per task, so bills still rise
- **Gartner predicts** GenAI cost per resolution in customer service will exceed 3 USD by 2030, more than many offshore human agents
- **So price the architecture** before you build it, and measure the unit the business pays for

Who deploys, who promotes, who pays depends on the mode

	Cohort mode	Solo mode	Local track
Platform	the instructor, once per track	you, in Part 0	<code>make local-up</code>
You are	a tenant, your handle on every resource	the tenant <code>solo</code>	the tenant in <code>.env</code>
Live target	shared; the instructor or the pipeline promotes	yours	yours, plus <code>higher</code>
Stop and destroy	the instructor	you, before you leave	<code>make local-down</code>
Cost	credit per learner: AWS 20 USD, Google Cloud 15, Azure 35	AWS about 74 USD, Google Cloud about 23, Azure about 30	32 GB on a Mac, or <code>SMALL=1</code> on 16 GB; about 60 GB of Docker disk

Route cheap first: Project 1 scores a ticket for nothing

```
triage = registry.execute("classify_urgency", {"subject": subject, "body": body})
if priority == "P0" and p0 >= policy.p0_confidence:    -> proposed escalation, no model call
if priority in policy.economy_priorities:              -> run_agent(role=ECONOMY, small caps)
else:                                                  -> run_agent(role=WORKHORSE)
```

Confident P0

A proposal to the duty manager, cost zero. The approval gate still applies, and the approval runs it once.

Economy

Nova Micro on AWS; gpt-oss-120b with Economy caps on Google Cloud; [mistral-small-2503](#) on Azure; gpt-oss-20b on Local.

Everything else

The Workhorse loop from the agent part, screener included.

The capstone is two numbers on the board, and the gateway's bill beside them

- 1 Read your key's `spend` on the gateway
- 2 The same twenty tickets through the plain loop and through your router
- 3 Resolved count, cost per resolved ticket, mean steps, the router's branches
- 4 Read `spend` again; explain any gap with the meter

Write yours, not a slide's

The split between P0 direct, Economy and Workhorse depends on your Project 1 model and its threshold.

Dollar figures on the delivery week's model come from the room.

Promotion is an approval, a stage change with a reason, and a canary

	Approval	Live target and canary	Rolls back
AWS	<code>APPROVED</code> event deploys your serverless endpoint	<code>northwind-live-triage</code> , SageMaker blue/green, 10 percent for 5 minutes	automatically, on the 5xx, p95 or quality level alarm
Google Cloud	alias moves; your Cloud Run service rolls a revision	endpoint <code>northwind-live-triage</code> , traffic split at 10, then finish	a person, paged by the alert policies
Azure	<code>stage</code> tag; your managed online endpoint takes the version	blue and green at 10 percent, then <code>promote</code> ; the instructor or the pipeline promotes live	a person, paged by the alerts
Local	alias <code>approved</code> in MLflow	<code>serving-canary</code> at weight 10, then <code>live</code>	a person, with <code>make local-canary WEIGHT=0</code>

Code promotes through the delivery pipeline: signed once, verified, approved by a person

	Starts the release	Signed and verified	Human approval	Canary
AWS	a push to <code>main</code> : CodePipeline via CodeConnections	AWS Signer with Notation; immutable ECR tags	the <code>Approve</code> stage	policy alias via CodeDeploy, 10 percent for 15 minutes
Google Cloud	<code>make release-gcp</code> or the trigger	cosign in Cloud Build, released by digest	<code>make approve-gcp</code> , then again to advance	Cloud Deploy on Cloud Run at <code>canary_percent</code>
Azure	<code>make release-azure</code> or Azure Pipelines	cosign with a Key Vault key	<code>make approve-azure</code> or the environment check	a Container Apps revision at 10 percent
Local	<code>make local-up</code>	not signed; local registry	you	<code>make local-canary WEIGHT=10</code>

Only AWS rolls a canary back on its own; everywhere else a person does

- **AWS:** both canaries roll back on the 5xx, p95 and quality level alarms. Drift and the other quality signals page
- **Google Cloud:** alert policies page a person; `-to-tags stable=100` or abandon the rollout
- **Azure:** alerts page a person; `make approve-azure` refuses while a live alert fires
- **Local:** no alarm moves traffic; `make local-canary WEIGHT=0` by hand

A canary needs traffic to judge

At 10 percent and about 8 requests a minute, 200 canary requests take four hours.

So the release sends the golden cases through the canary while it bakes, or bakes at 50 percent, and says which.

A model can be up and wrong, so quality has alarms too

Signal	Service	Bar
Quality level	every service	2 is an alert; the only quality signal that stops a canary
Shadow agreement	triage, semantic	below 0.90 over at least 200 shadowed requests
P0 share ratio	triage, semantic	2 or above, or 0.5 or below, outside the 99 percent interval
Refusal ratio	policy	2 or above, outside the 99 percent interval
Sampled judge score	agent	below 3.5 over at least 20 judged runs

Each monitor logs `quality_alert` with the signal, the value, the bar and the window. Alarms: `<env>-<svc>-quality-{level,shadow,p0-high,p0-low,refusal,judge}` on AWS and Azure; "quality ... past its bar" on Google Cloud.

AI systems degrade silently; the panels make it loud

A model answers slower

Nothing errors. Users wait.

p95 latency

A retriever returns nothing

The model refuses, or answers from nothing.

Refusal ratio

The agent spends more

More steps, more tokens, the same answers.

Cost per resolution

AWS: the CloudWatch dashboard, metrics as Embedded Metric Format lines, traces in X-Ray. Google Cloud: the `northwind` dashboard from log-based metrics, Cloud Trace. Azure: the platform workbook, Application Insights. Local: Grafana on :3000, Prometheus, Jaeger, Evidently.

An SLO is a ratio over 28 days, and it pages on how fast the budget burns

SLI	SLO, 28 days
Availability, triage and semantic	99.5 percent without a 5xx
Availability, policy and agent	99.0 percent
Latency, triage and semantic	95 percent under 2 s, warm
Latency, policy	95 percent under 8 s, warm
Cost per resolution	at most 1.5 times the track baseline

Alert	Burn rate	Windows
Page	14.4	1 hour and 5 minutes
Ticket	6	6 hours and 30 minutes

While the budget is spent: promotions freeze, gates stay, and the operator's controls go in order: kill switch, concurrency cap, spend cap.

A key rotates per owner, never during a traffic split, and is printed once

```
make rotate-key TRACK=aws TENANT=<handle> DRY_RUN=1  
make rotate-key TRACK=aws TENANT=<handle>
```

Per owner

`live` and each tenant have their own key; `TENANT=all` walks them one at a time.

Refuses mid-canary

A split would accept the new key on one slice and the old on the other. The script stops while a Lambda alias, a Cloud Run service or a Container App splits traffic.

Old keys go last

Old versions are disabled only after every service of the owner serves the new one. The new key is printed once and stored nowhere else.

EU accounts route to EU models, or the call fails before it leaves

Role	AWS	Google Cloud	Azure
Workhorse	EU id in <code>eu-central-1</code>	no EU option	<code>Mistral-Large-3</code> on Data Zone EU
Economy	EU id	no EU option	<code>Mistral-Large-3</code> on Data Zone EU
Judge	<code>eu.anthropic.claude-opus-5</code>	Claude on the <code>eu</code> multi-region	no EU option

A missing EU option raises `ResidencyError` before any network call. There is no silent fallback to another region.

A higher environment is the same code, a second target and one role that may cross

A second target

A Deploy action per environment in CodePipeline; a second Cloud Deploy target; a second resource group on Azure; the `higher` profile locally. Each with its own approval and canary.

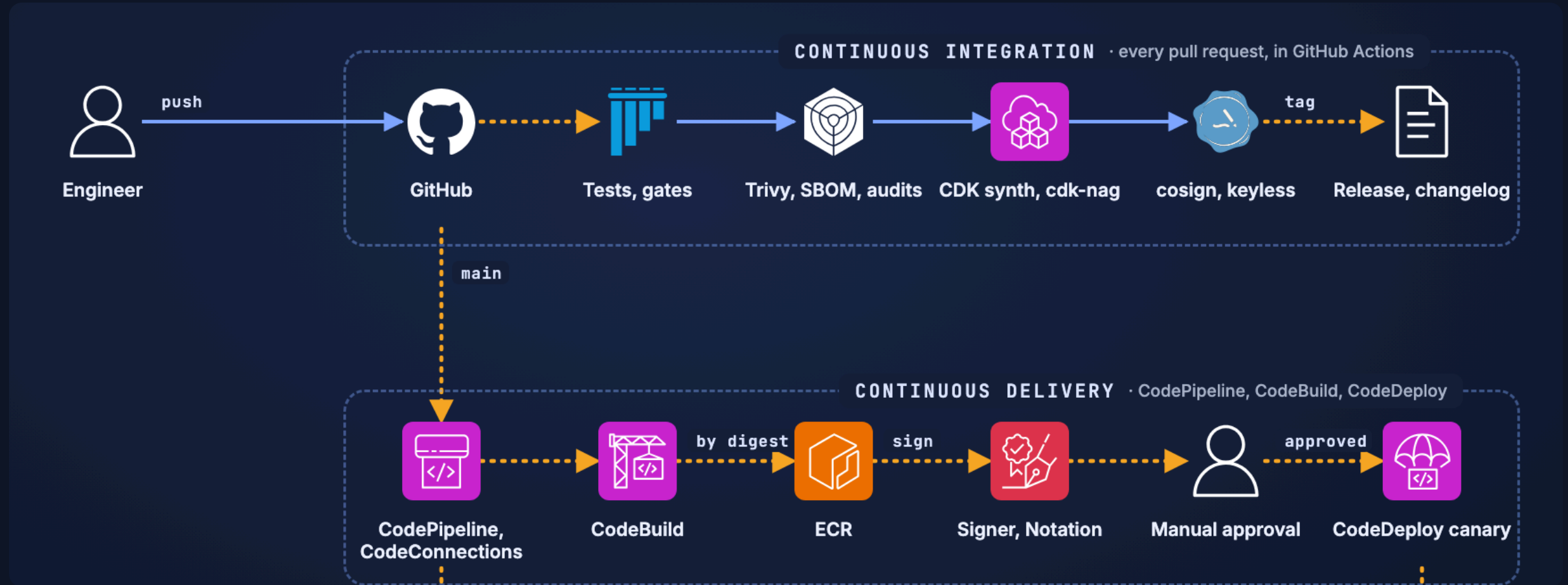
A deployment role

`northwind-<env>-deployer`, a deployer service account or a federated deployment identity that trusts exactly one principal: the pipeline's role below.

Promotion by reference

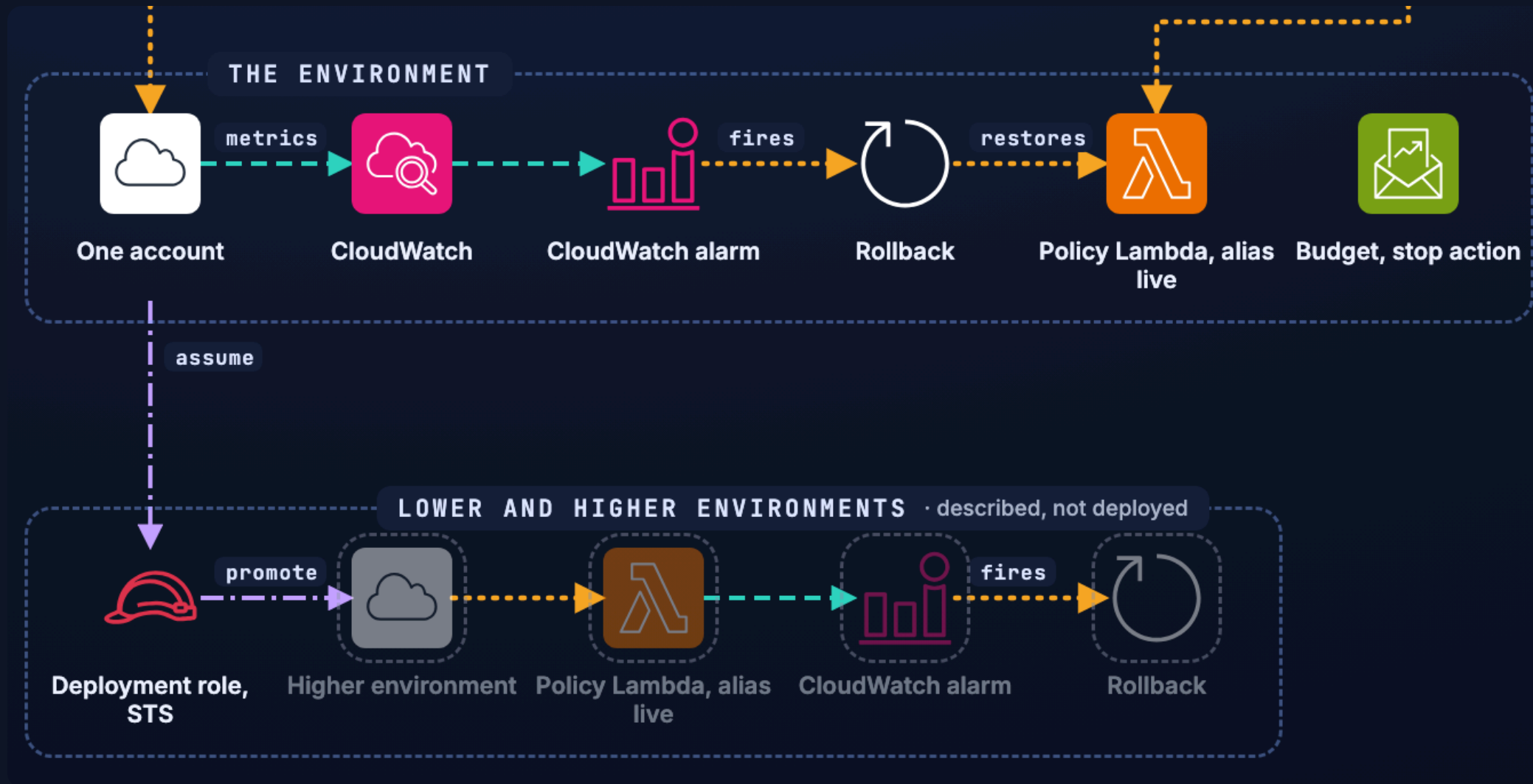
Images by digest, model versions copied with the same artifact URI. Never a rebuild, never a retrain.

DevOps, AWS: from a pull request to a signed image



Continuous integration in GitHub Actions: tests and gates, Trivy, the SBOM and audits, CDK synth with cdk-nag, cosign keyless, the release. A merge starts CodePipeline: CodeBuild, ECR by digest, Signer with Notation, the manual approval, the CodeDeploy canary.

DevOps, AWS: the one environment, and how the next one is reached



CloudWatch alarms fire and roll the policy alias back; the budget's stop action. Below, dashed: a deployment role assumed through STS into a higher environment, described and not deployed.

The whole system, AWS: delivery into the canary, and what the operator sees



From the system figure: Secrets Manager, GitHub Actions, CodePipeline, ECR with signed images, the DNS Firewall on egress, the WAF priced but not deployed; the canary on the alias, X-Ray, the kill switch, the dashboard and SLOs.

Acceptance criteria

1. The router's three branches pass; a confident P0 costs nothing; its card lists both roles
2. Two costs per resolved ticket and the gateway spend in `NOTES.md`
3. An approved version live behind a canary, or watched in cohort mode, with the reason
4. Your track's release path and rollback command written down
5. One SLO row with its panel; the path to a higher environment drawn

57 then 45

`make session06` : the capstone tests, then the AWS synth review when `deploy/aws/.venv` exists

Questions worth asking

- Of every failure across the six parts, which would have reached production in your setup, and which control would have caught it?
- Your router saves money on P0 and Economy tickets. What happens when Project 1's P0 recall drops, and which alarm tells you first?
- Which identity in your organisation can write to production without going through a pipeline?
- If the model vendor changed tomorrow, what in this repo would you touch?

RESERVE

Appendix

The organisation's landscape, the whole system region by region, and the other tracks' figures. Use a slide when the room asks for it.

Three frameworks give governance a shape an auditor recognises

NIST AI RMF 1.0

Four functions: **Govern, Map, Measure, Manage**. Released 2023-01-26; NIST says a revision is under way.

NIST AI 600-1

The Generative AI Profile, 2024-07-26: **12 risks** GenAI creates or worsens, among them confabulation, data privacy, information security and value chain integration.

ISO/IEC 42001:2023

The first international **management system** standard for AI, and certifiable. AWS announced certification in 2024-11 for Bedrock and three other services.

[docs/governance/](#) holds a start on the evidence: a DPIA, a risk register mapped to NIST AI 600-1, datasheets, a retention and erasure runbook, a residency table. It is not a certification.

AI TRiSM is a checklist: every control in this course sits in one of four layers

AI governance

Use case catalog, agent registry, model and prompt registries, calibrated Judge, tiered gates.

Runtime inspection and enforcement

Guardrails, the Cedar policy, the approval gate, quality alarms, kill switch, spend caps.

Information governance

Redaction, the audience from the key, residency routing, data classes per use case.

Infrastructure and stack

Least privilege per tenant within each cloud's limits, secrets by reference, scanning, SBOM, signing.

<https://www.gartner.com/en/articles/ai-trust-and-ai-risk>

Value follows the operating model more than the model

10 / 20 / 70

where BCG's AI leaders put their effort: algorithms, data and technology, then people, processes and culture

Where the course stands

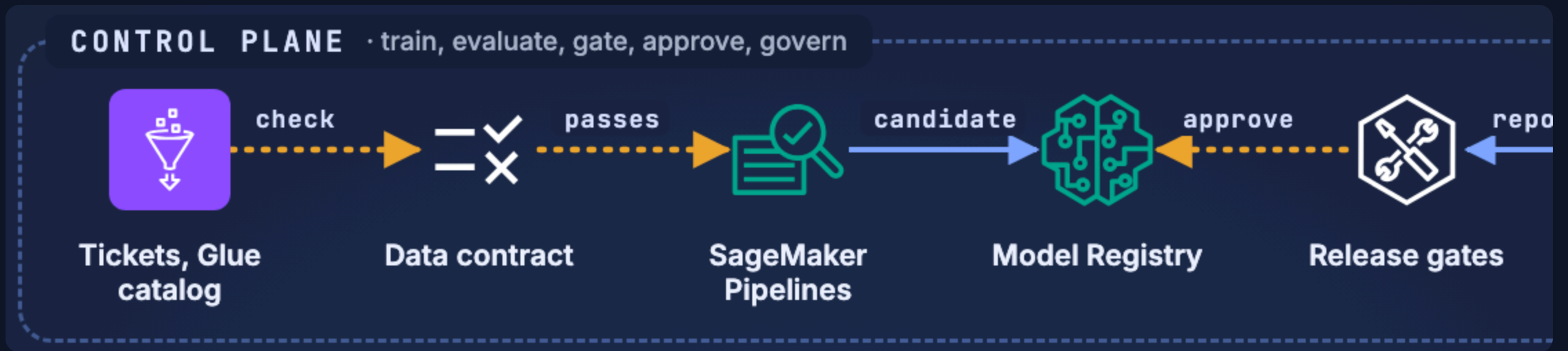
The MLOps levels from the machine learning part: the course builds the **level 1 pieces** for Projects 1 and 2 (a gated pipeline, a registry, an approval, monitoring). It does not claim level 2.

A team with the whole platform that still promotes by hand is at level 0.

The ladder rewards scope, so choose your next depth on purpose

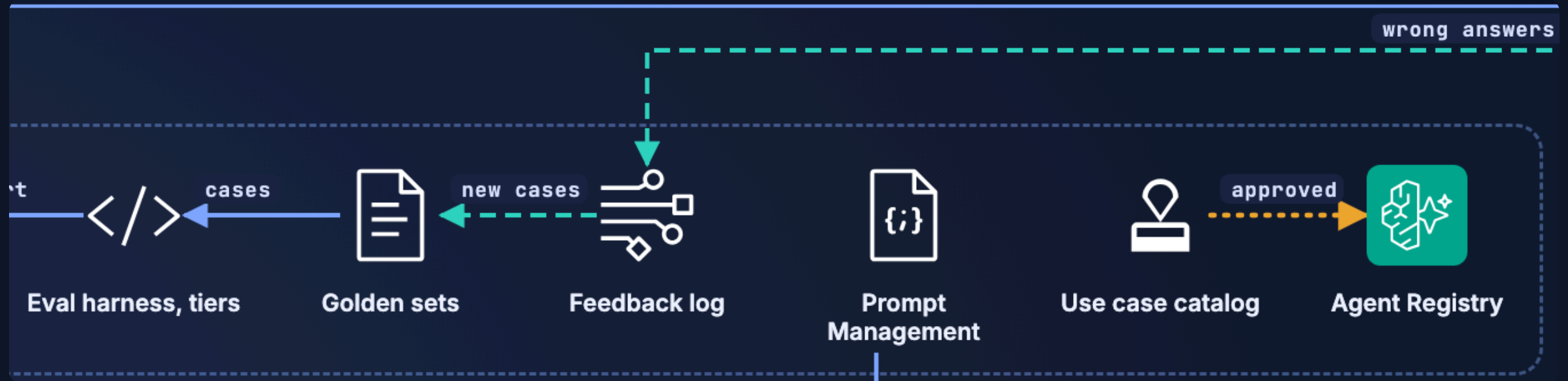
Growing toward	Learn next	Evidence you already have
Senior AI engineer	evaluation design, cost modelling, one serving stack in depth	the Project 3 harness, the capstone's cost numbers
Agentic AI engineer	tool design, agent evaluation, MCP and A2A security	the tiered gate, the approval that runs once
MLOps and platform	multi-tenant gateways, delivery, SLOs	the promotion drill, the SLO row
AI safety and governance	NIST AI RMF, ISO/IEC 42001, the EU AI Act in practice	the catalog, docs/governance/

The whole system, AWS: the control plane, from data to gates



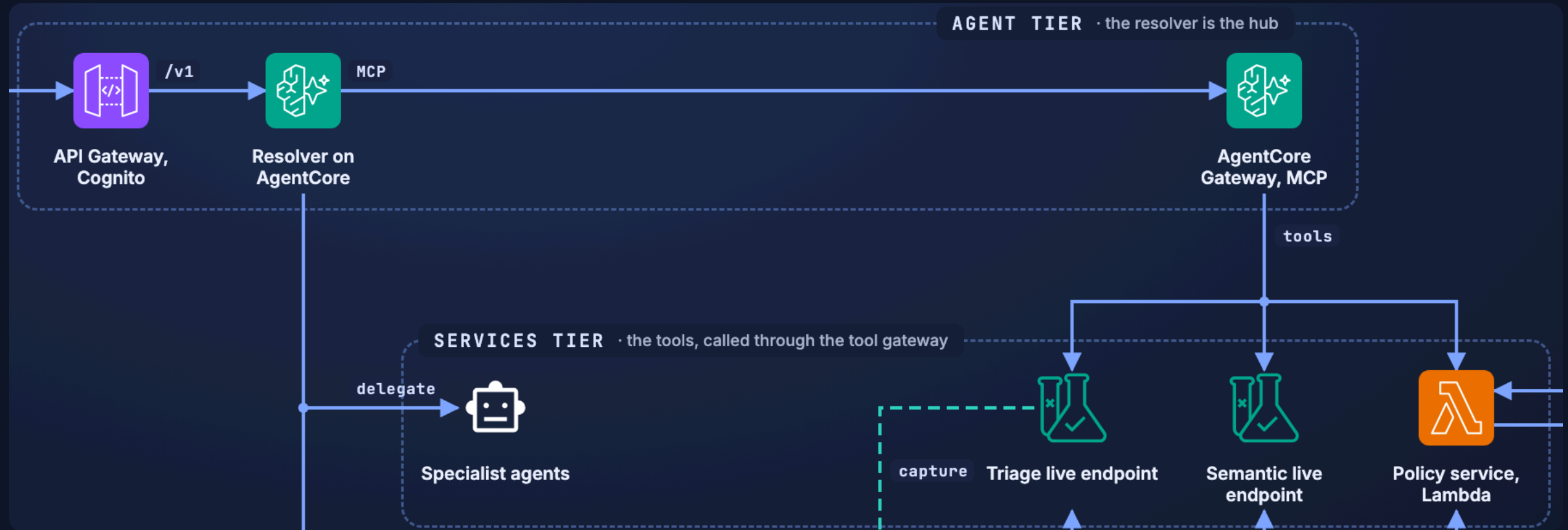
Tickets in the Glue catalog, the data contract, SageMaker Pipelines, the Model Registry with an approval, the release gates.

The whole system, AWS: the control plane, from evaluation to registries



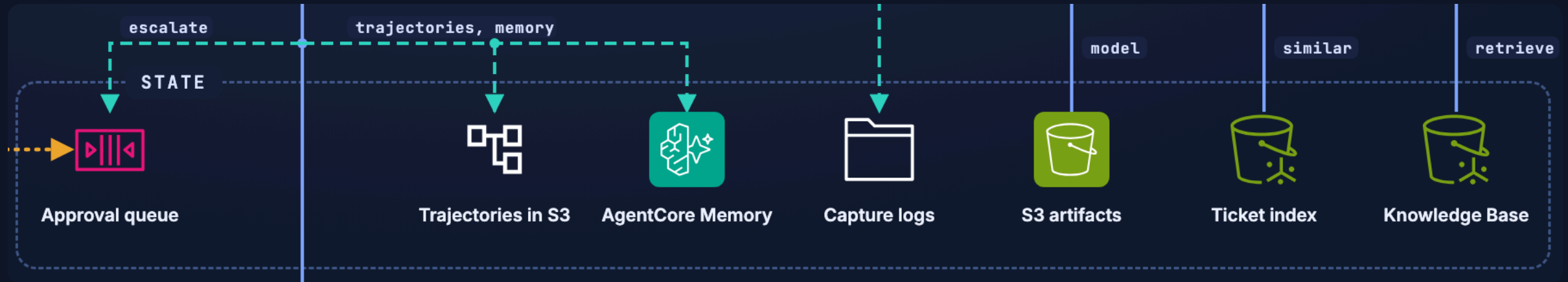
The eval harness by tier, the golden sets fed by the feedback log, Prompt Management, the use case catalog and Agent Registry.

The whole system, AWS: the agent tier and the services it calls



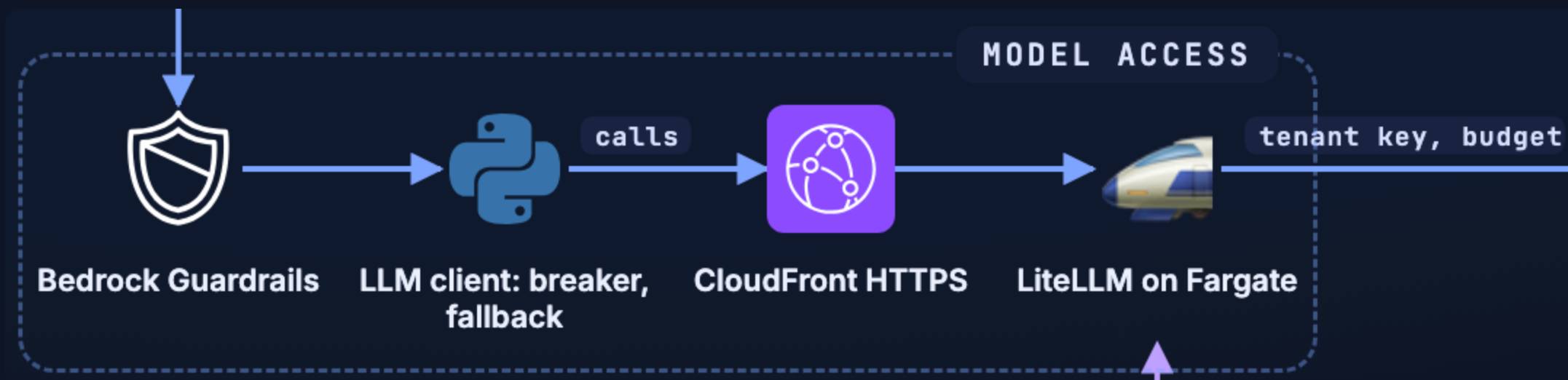
The ticket through API Gateway with Cognito to the resolver on AgentCore; specialists by delegation; tools through AgentCore Gateway over MCP to the triage and semantic live endpoints and the policy Lambda.

The whole system, AWS: state



The approval queue, trajectories in S3, AgentCore Memory, the capture logs, the S3 artifacts, the ticket index and the Knowledge Base.

The whole system, AWS: model access



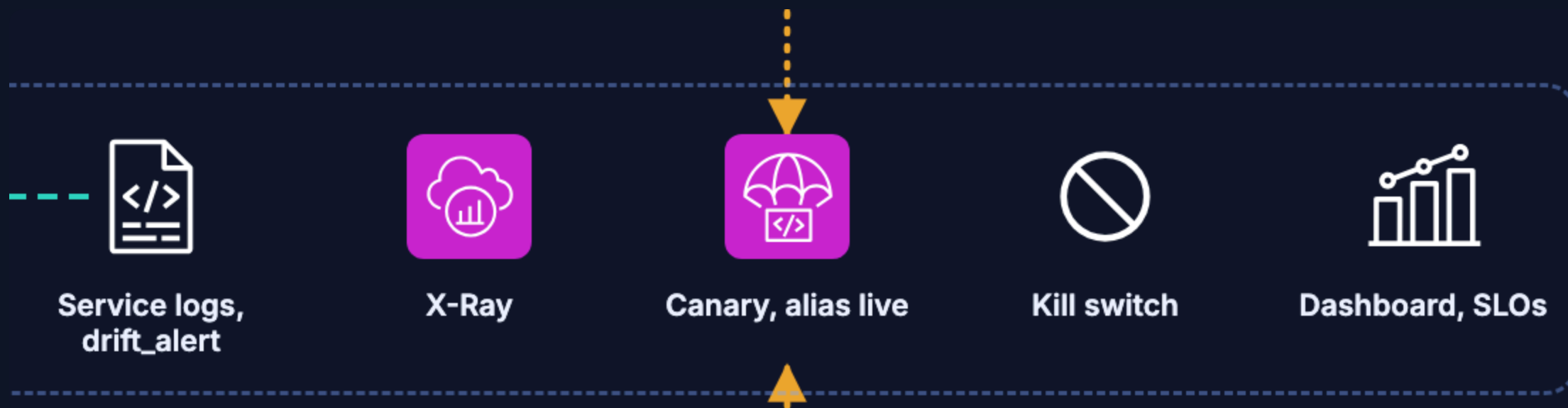
Bedrock Guardrails, the LLM client with its breaker and fallback, CloudFront HTTPS, LiteLLM on Fargate, on the tenant's key and budget.

The whole system, AWS: operations, from logs to the page



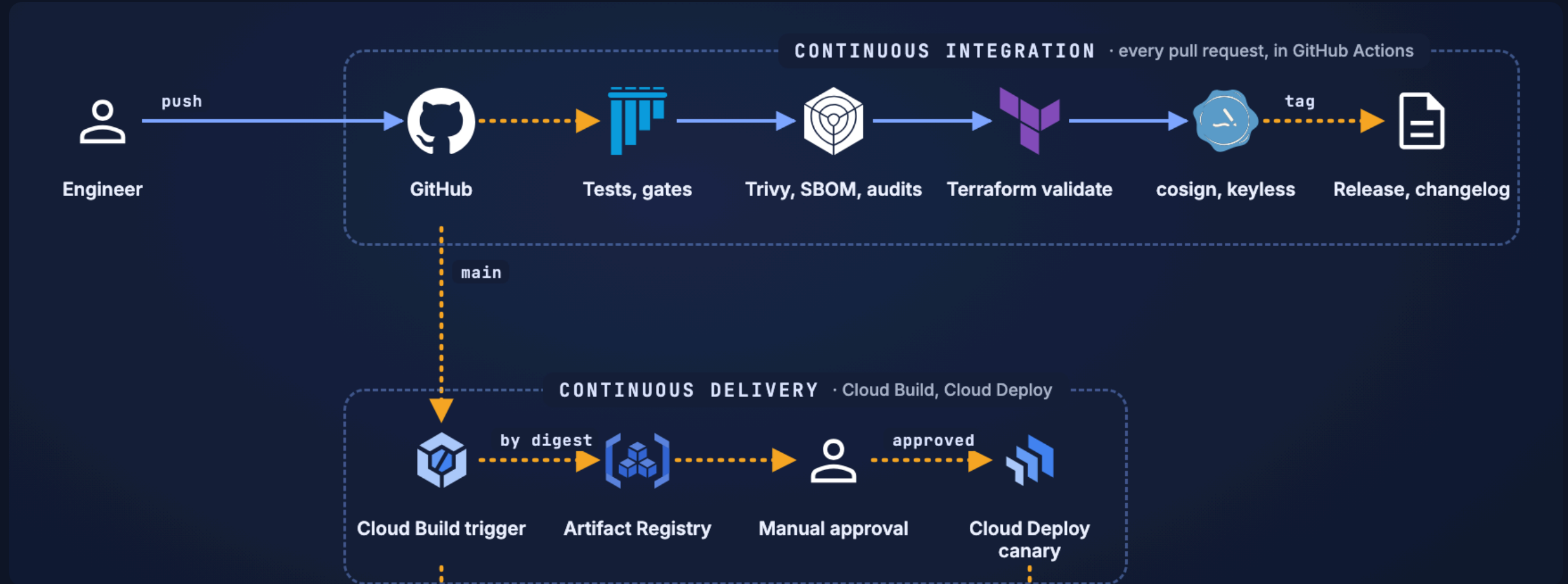
The budget's stop action attaches a deny policy; SNS pages on call; CloudWatch alarms read metric filters and EMF over the service logs and their drift_alert lines.

The whole system, AWS: operations, from the canary to the dashboard



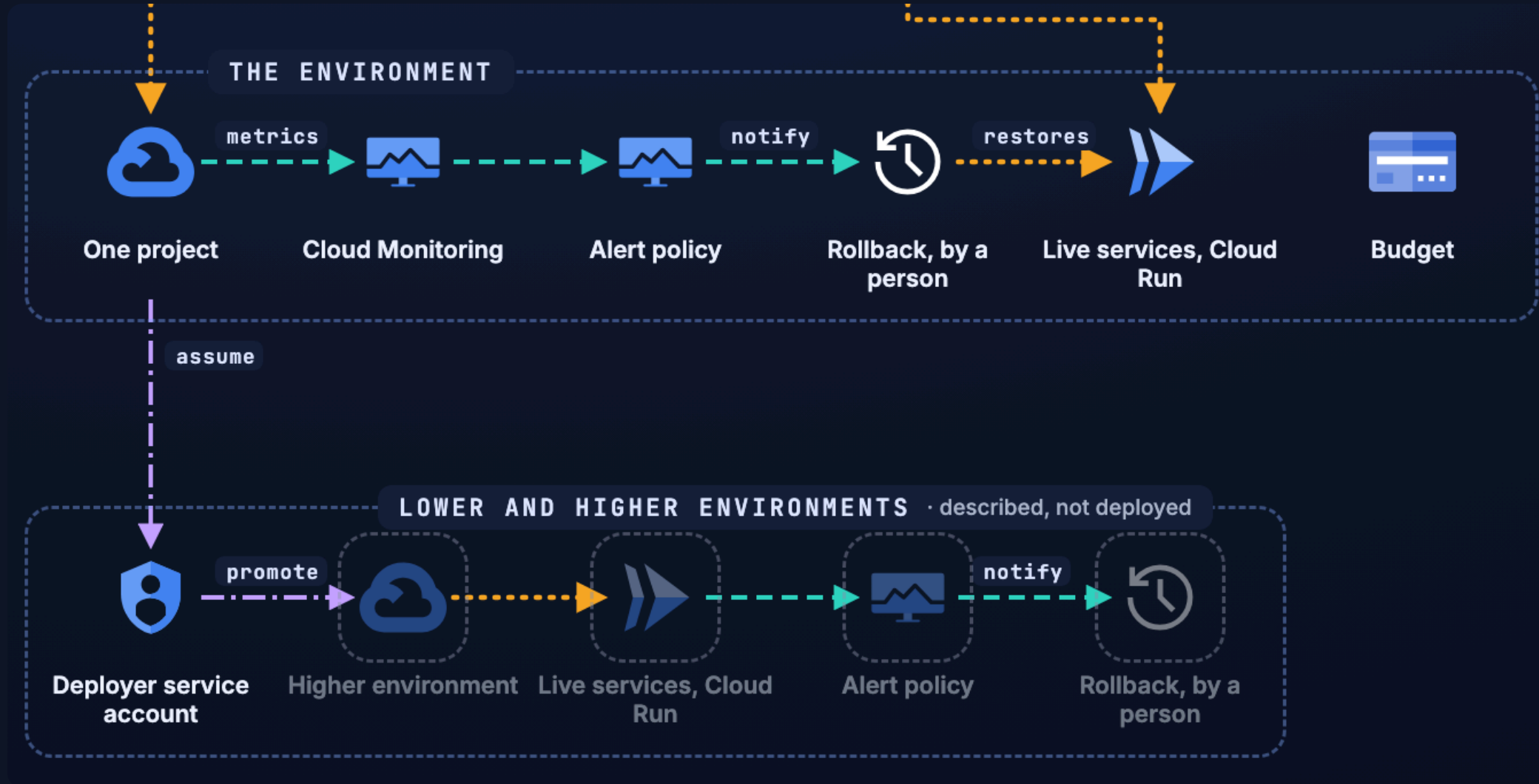
X-Ray, the canary on the alias, the kill switch, the dashboard and SLOs. Below this crop the figure draws the line from the alarms back to the canary.

DevOps, Google Cloud: from a pull request to a released digest



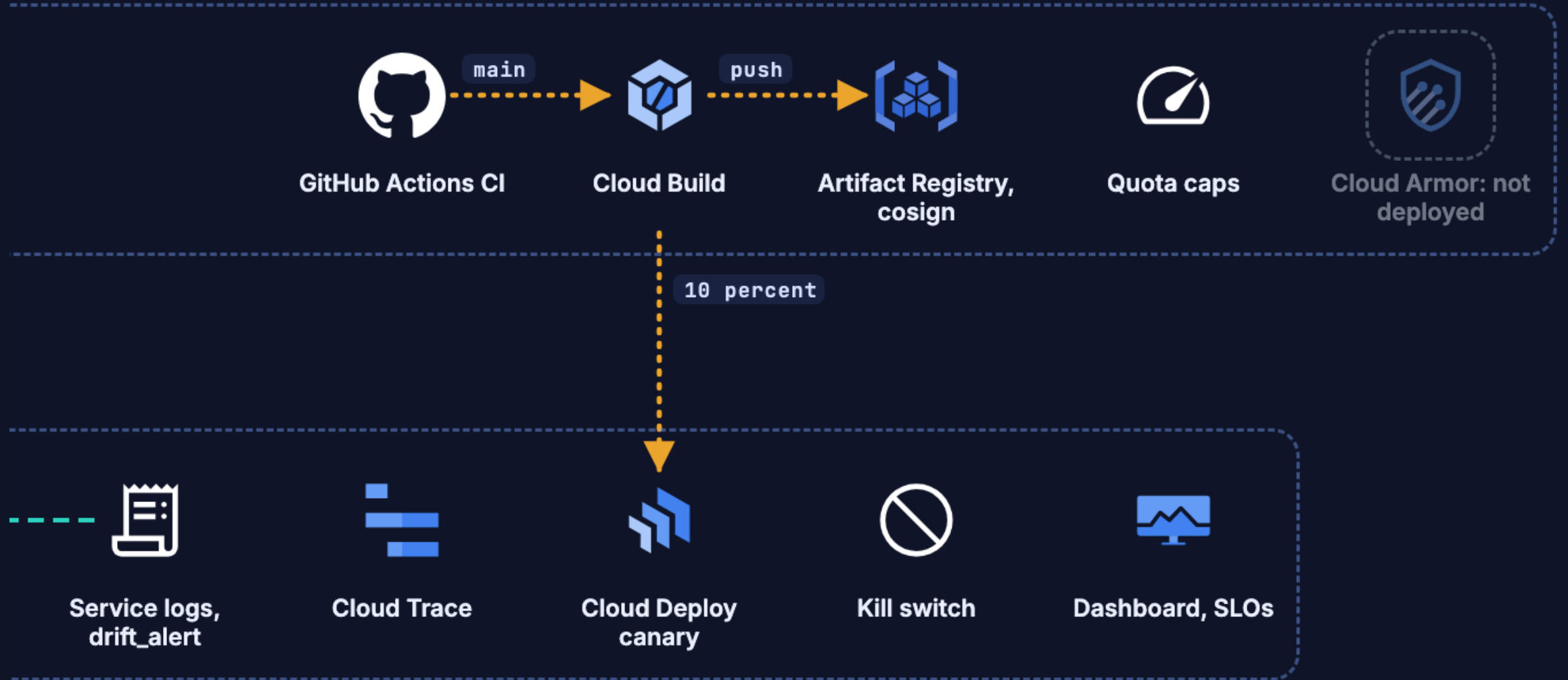
The same pull-request checks with Terraform validate; Cloud Build, Artifact Registry by digest, the manual approval and the Cloud Deploy canary.

DevOps, Google Cloud: the one project, and how the next one is reached



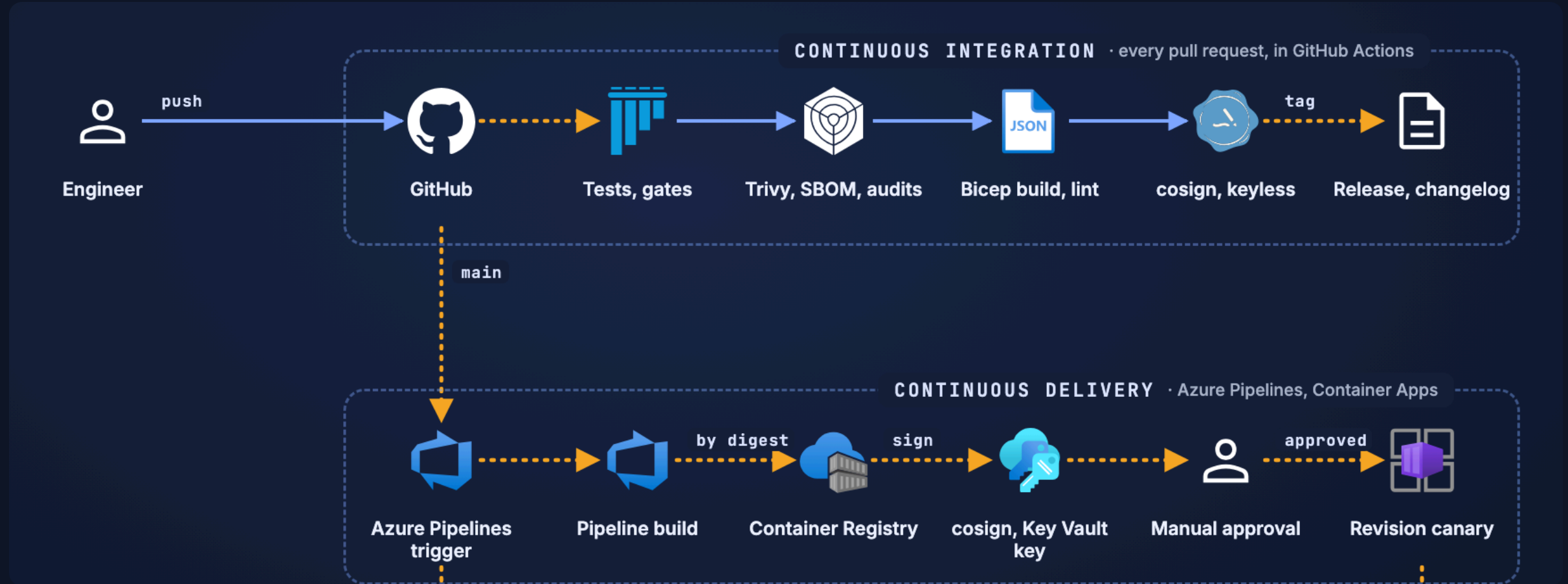
Cloud Monitoring, an alert policy, a rollback by a person, the live services on Cloud Run; a deployer service account into a higher environment, described and not deployed.

The whole system, Google Cloud: delivery and operations



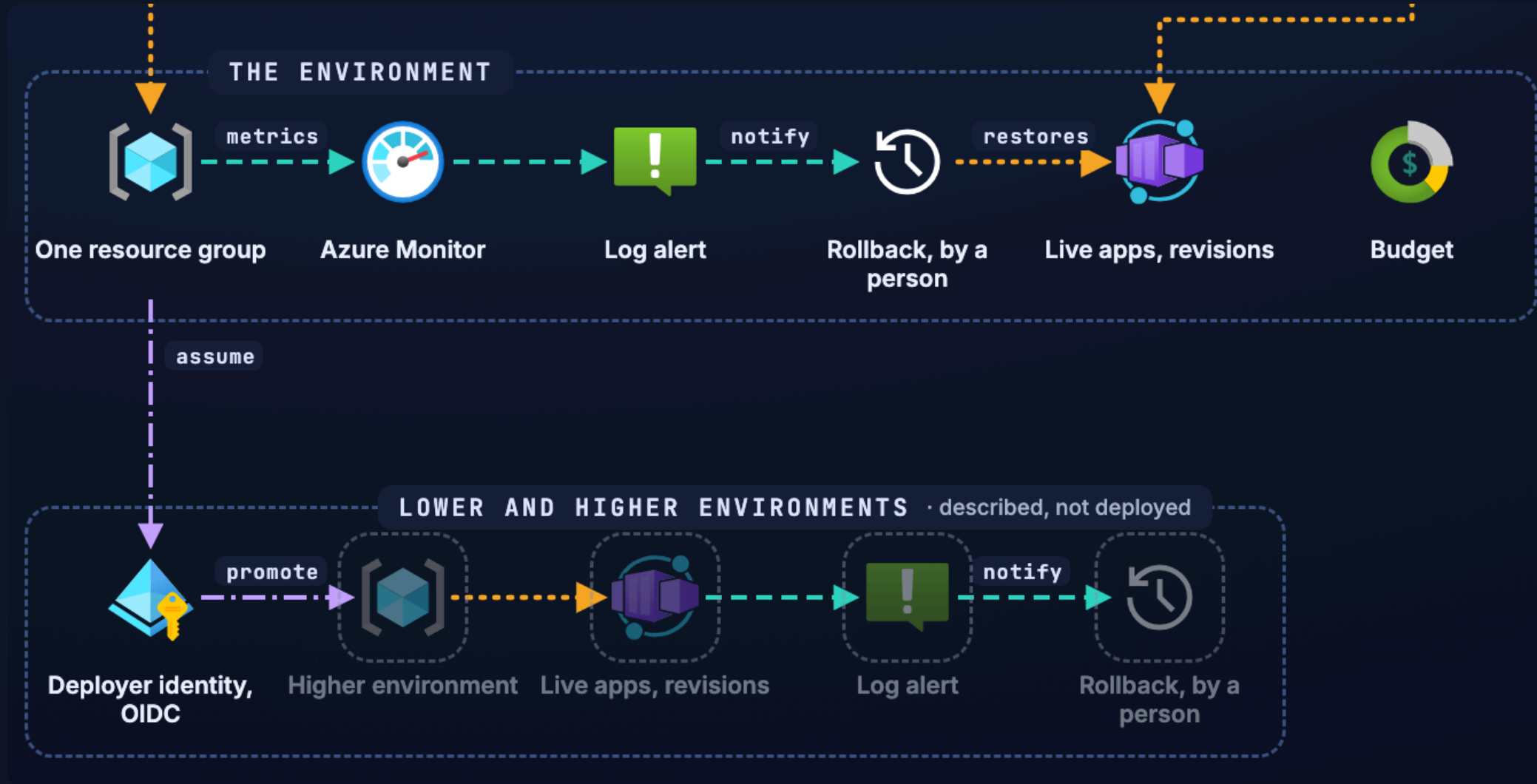
GitHub Actions, Cloud Build, Artifact Registry with cosign, quota caps, Cloud Armor not deployed; the service logs, Cloud Trace, the Cloud Deploy canary, the kill switch, dashboards and SLOs.

DevOps, Azure: from a pull request to a signed revision



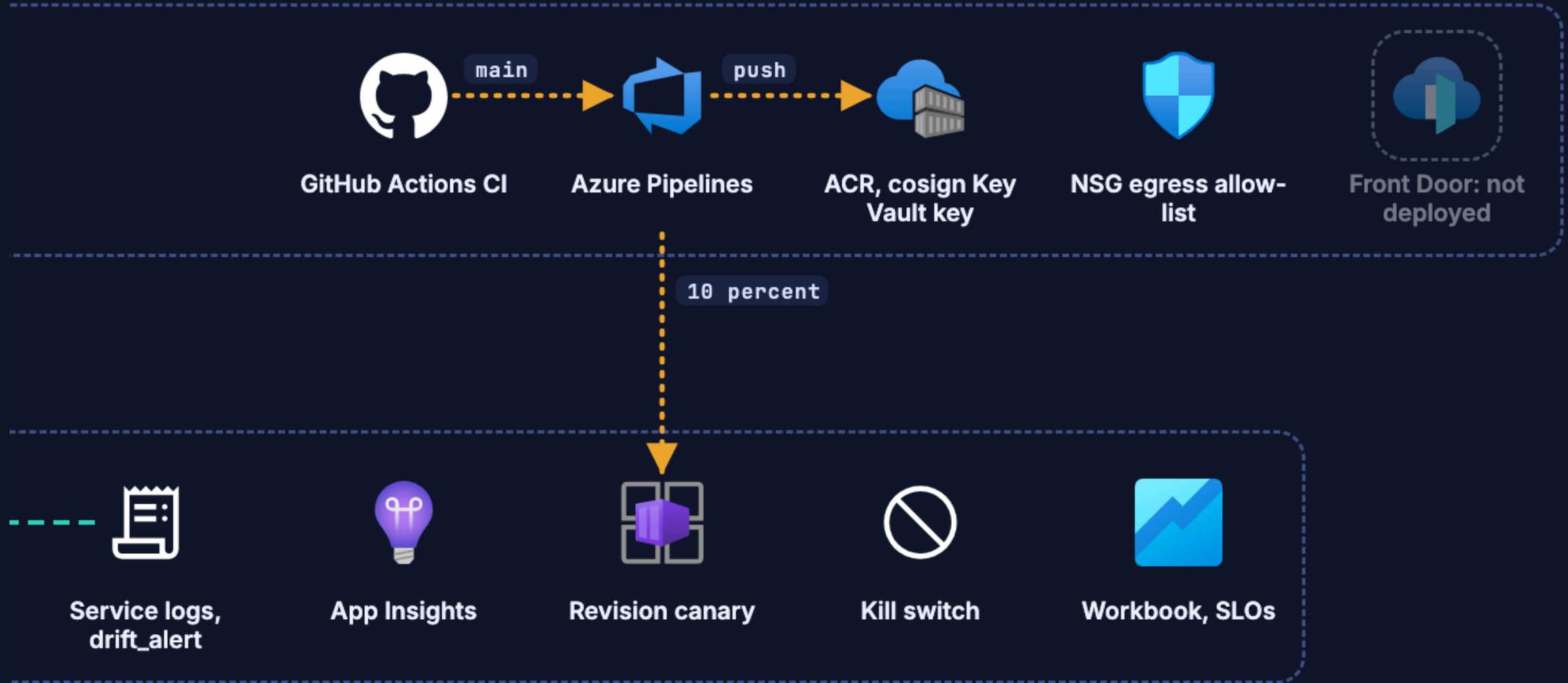
The same checks with the Bicep build and lint; Azure Pipelines, Container Registry by digest, cosign with a Key Vault key, the manual approval, a revision canary.

DevOps, Azure: the one resource group, and how the next one is reached



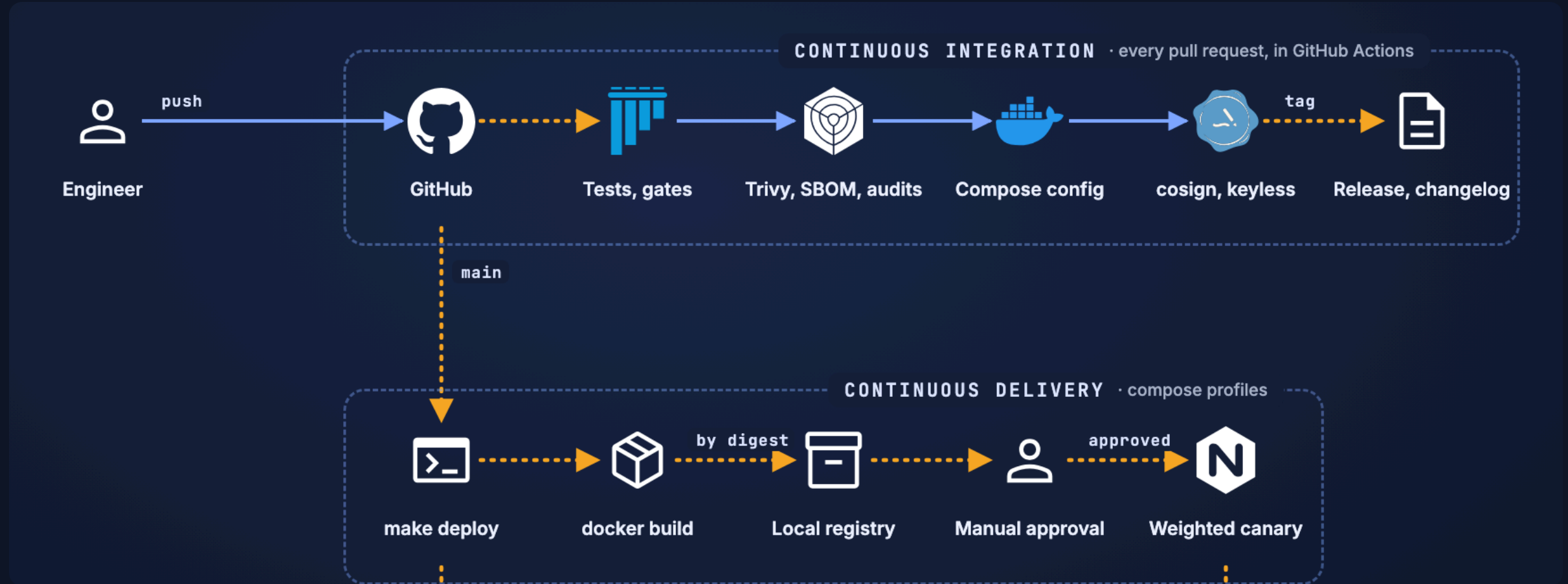
Azure Monitor, a log alert, a rollback by a person, the live apps and revisions, the budget; a federated deployer identity into a higher environment, described and not deployed.

The whole system, Azure: delivery and operations



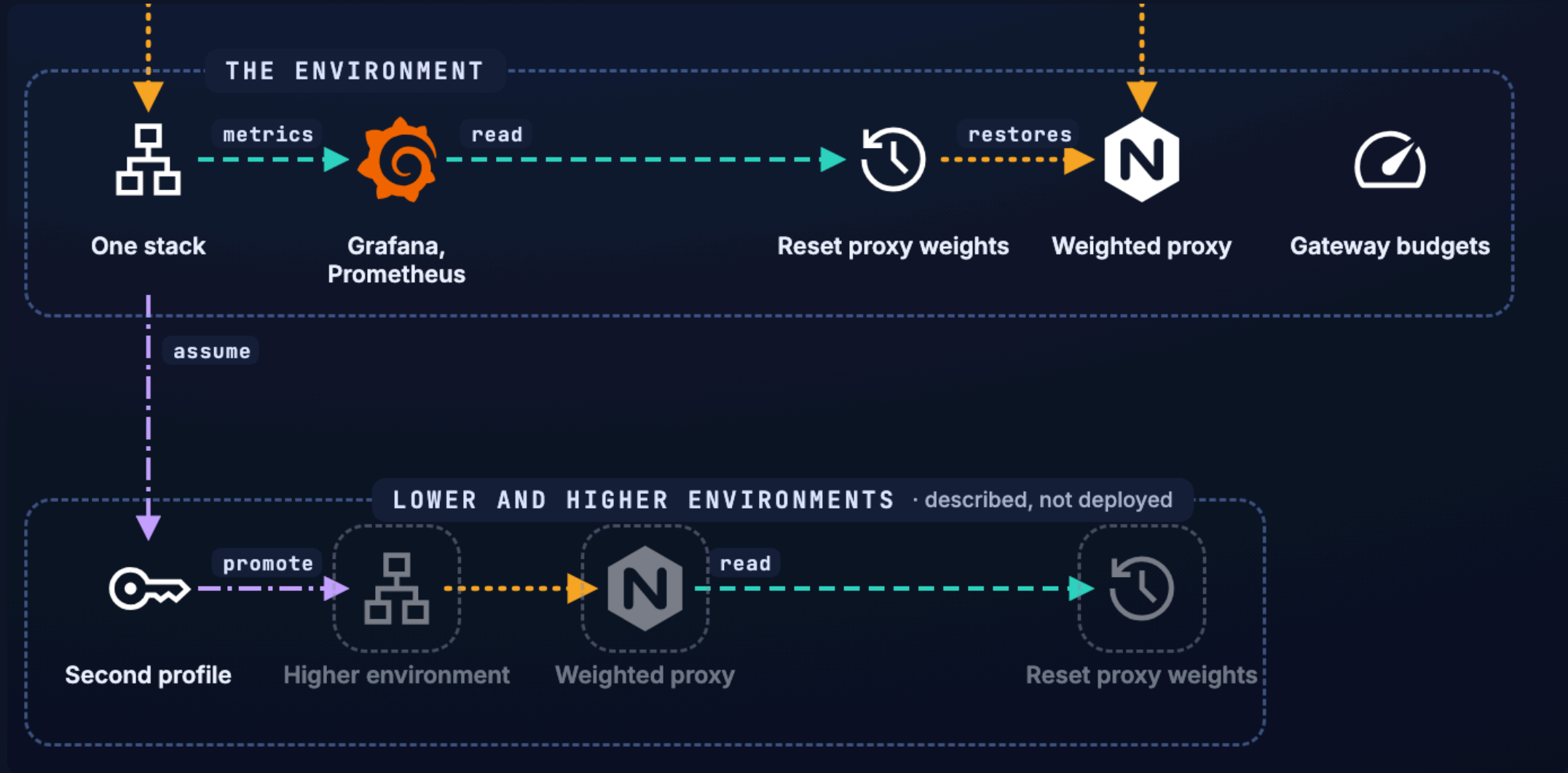
GitHub Actions, Azure Pipelines, the registry with a cosign Key Vault key, the NSG egress allow-list, Front Door not deployed; the service logs, Application Insights, the revision canary, the kill switch, the workbook and SLOs.

DevOps, Local: from a pull request to the local registry



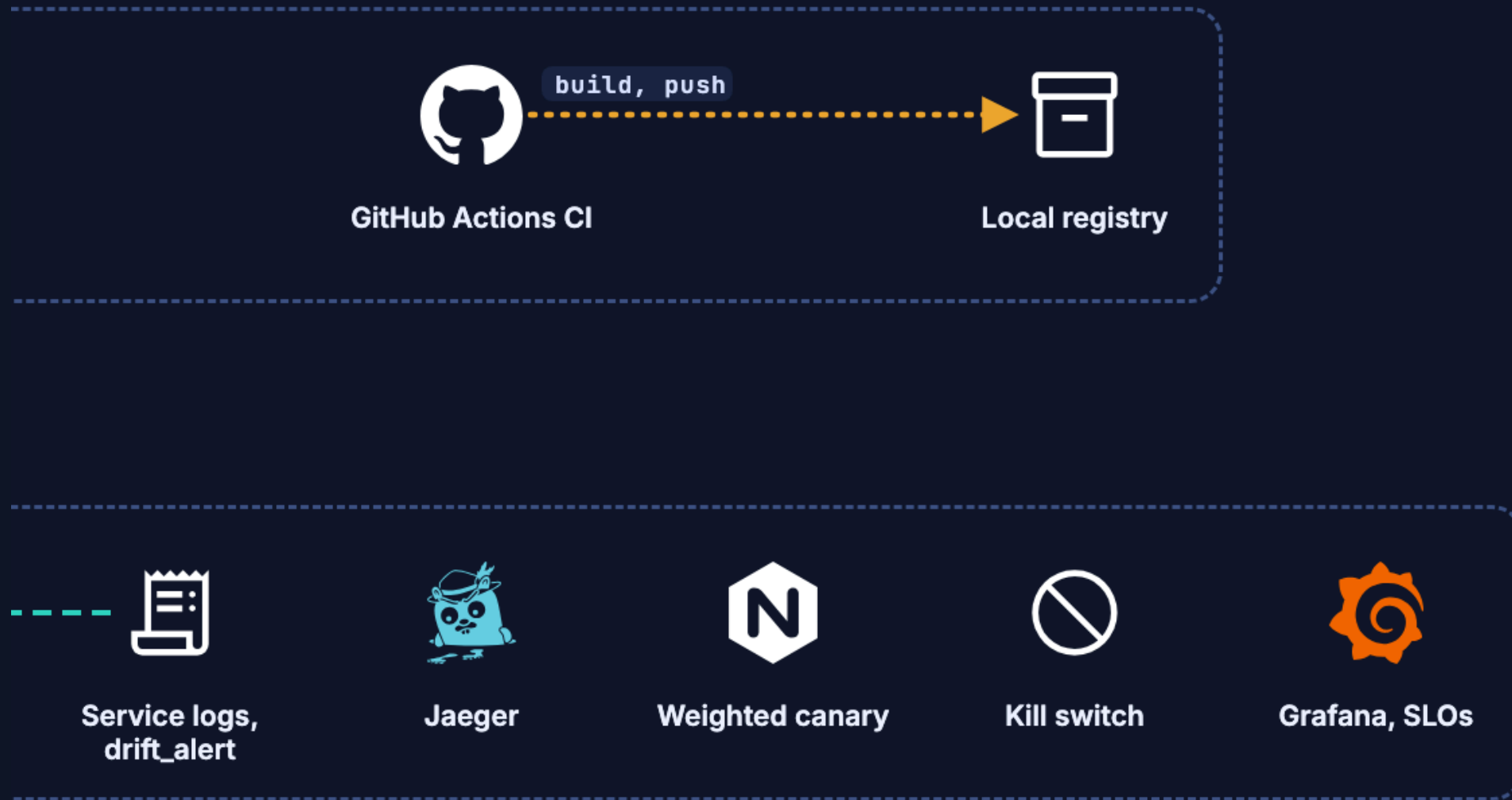
The same checks with the compose config; make deploy, docker build, the local registry by digest, a person, the weighted canary.

DevOps, Local: the one stack, and the higher profile



Grafana and Prometheus, a person resets the proxy weights; a second profile as the higher environment.

The whole system, Local: delivery and operations



GitHub Actions building and pushing to the local registry; the service logs, Jaeger, the weighted canary, the kill switch, Grafana and SLOs.